Kristiania

Benchmarking and Complexity Analysis of Fundamental Sorting Algorithms

Course: PG4200-H: Data Structures and Algorithms

Group number: 44

Candidate numbers: 203 & 184 **Exam 2026**

1. Introduction **3**
2. Benchmarking **3**
2.1 Dataset **3**
2.2 Benchmark Methodology **3**
2.3 JVM Warm-up **4**
3. Task 1 – Bubble Sort **5**
3.1 Implementation 5
3.2 Time Complexity 6
3.2.1 Calculating worst-case time complexity 7
4. Task 2 – Insertion Sort **9**
4.1 Implementation 9
4.2 Time Complexity 9
4.2.1 Calculating worst-case time complexity 10
5. Task 3 – Merge Sort **11**
5.1 Implementation 11
5.2 Number of merges on shuffled- and raw dataset 12
6. Task 4 – Quick Sort **12**
6.1 Implementation 12
6.2 Comparisons and pivot strategies 14
7. Conclusion **16**
8. Additional Notes **17**
8.1 Task 3 – Extra Note on Merge Sort Time Complexity 17
8.2 Task 4 – Extra note on Quick Sort Time Complexity 18
8.3 Space Complexities 19
8.4 Program architecture 19
8.5 Reflections 23
9. Bibliography **23**

1. Introduction

This report documents the implementation and analysis of four fundamental sorting algorithms: Bubble Sort, Insertion Sort, Merge Sort, and Quick Sort. Each algorithm is implemented in Java and evaluated both theoretically through time-complexity analysis, and empirically through benchmarking on a real dataset of wine quality measurements.

In addition to answering all problems posed in the exam, we will attempt to conclude with which of the algorithms explored is best suited to sort the raw dataset.

This report also contains an "Additional Notes"-section where we explore further beyond the scope mandated by the exam, such as the time complexities of the quick- and merge-sort algorithms, the space complexities of all explored algorithms and our program architecture.

Quick note on notation: Θ (Theta) is often simplified as "average case", Ω (Omega) as "best case", and O (Big-O) as "worst case" but strictly speaking these are mathematical bounds. Theta means a tight bound (both above and below), Omega means a lower bound, and Big-O means an upper bound. We employ these terms in their simplified meaning when describing benchmarking results. This is consistent with the style employed in the course materials as well.

2. Benchmarking

2.1 Dataset

The dataset is acquired from: https://archive.ics.uci.edu/dataset/186/wine+quality. (Paulo Cortez, 2009) We pre-process the data, combining the red and white wine entries and filtering out all non-unique alcohol content values resulting in 111 entries. This is referred to as the wine dataset or the dataset. For each algorithm we examine how the dataset input order: RAW, SHUFFLED, and SORTED affects the number of operations and time elapsed. In some cases we also benchmark a larger dataset (wine dataset including duplicates) which consists of 6497 entries, in order to demonstrate how input size affects time and operation-counts. This list is tagged “ALL” in the benchmarks.

The RAW dataset has an unmodified input-order. The SHUFFLED dataset employs the Collections.shuffle function to produce a shuffled list, and the sorted list is sorted in ascending order using Quick Sort with median pivot. Our report will justify this choice later in section: 7. Conclusion. The same lists are used in each benchmarking test per program cycle, to ensure a consistent testing-scenario for each algorithm.

2.2 Benchmark Methodology

Each algorithm is run 100 times on each dataset and input-order combination, to capture the average results instead of outliers. The results dissected in this report is included as a file in the project root folder: *reference_benchmark_results.txt*

Running the program several times will produce different ‘winners’ in certain scenarios based on the time-metric. However, measuring the algorithms with the granularity of nanoseconds will unavoidably contain noise from variables we cannot predict, such as environmental factors like JVM garbage collection and OS scheduling. We therefore consider both time measurements and average operations performed when evaluating the algorithms. Avg. operations performed are more reliable than time-measurements since they are deterministic and hardware-independent, however operation counts are not comparable across different algorithms as we count different kinds of operations (swaps, comparisons and merges). When we choose the optimal algorithm for sorting our RAW dataset later, we will consider time, average operations performed and relevant theory to make the decision.

2.3 JVM Warm-up

Since our program evaluates algorithms on time, we need to account for variables that could pollute our results. One such variable is the JVM's built-in JIT (Just-In-Time) optimization strategy, that improves the efficiency of Java applications by compiling bytecodes to native machine code at run time.

“In practice, methods are not compiled the first time they are called. For each method, the JVM maintains an invocation count, which starts at a predefined compilation threshold value and is decremented every time the method is called. When the invocation count reaches zero, a just-in-time compilation for the method is triggered.” *(IBM, 2024)*

This means that without warmup, early benchmark runs execute as interpreted bytecode rather than compiled native code, measuring interpretation overhead rather than the algorithm's true performance. Our warmup routine runs each algorithm 10 times across all three input types before measurements begin, pushing each method past its compilation threshold so all benchmarking runs benefit from JIT-compiled execution.

We also make sure to run each warm-up right before the relevant algorithm, instead of in bulk at the start. This is in order to aim for the same optimization level for each algorithm. “A method can be re-compiled to a higher optimization level through different mechanisms. One of these mechanisms is sampling: the JIT compiler maintains a dedicated sampling thread which wakes up periodically and determines which Java methods appear more often at the top of the stack. Such methods are deemed to be more important for performance, and they are candidates for being re-optimized at the higher levels of hot, veryHot, or scorching.” (*IBM*, 2024).

We are unable to reliably control which level of hot our algorithms will run at. These are just our attempts to alleviate the overall noise which will inevitably present in our time measurements.

```
// Problem 1 - Bubble Sort
System.out.println("--- | Warming up Bubble Sort");
BenchmarkHandler.jvmWarmup(warmupRounds, rawWines, sl   ← First warm-up bubble sort
BenchmarkHandler.jvmWarmup(warmupRounds, rawWines, sl
System.out.println("--- | Executing Bubble Sort benc
results.add(BenchmarkHandler.benchmark( name: "Bubble
results.add(BenchmarkHandler.benchmark( name: "Bubble
results.add(BenchmarkHandler.benchmark( name: "Bubble   ← Then perform bubble sort benchmarking
results.add(BenchmarkHandler.benchmark( name: "Bubble
results.add(BenchmarkHandler.benchmark( name: "Bubble
results.add(BenchmarkHandler.benchmark( name: "Bubble
System.out.println("--- | Bubble Sort benchmarking c

// Problem 2 - Insertion Sort
System.out.println("--- | Warming up Insertion Sort"   ← Then warm-up insertion sort (and so on... )
BenchmarkHandler.jvmWarmup(warmupRounds, rawWines, sl
```

Image: images/Warmup_Description.jpg

Code: src/main/java/main.java

3. Task 1 – Bubble Sort

3.1 Implementation

The non-optimized implementation compares each element to its neighbour, and swaps places if the current element is greater than the next. This emulates a bubble rising towards the surface of a liquid: the highest value “bubbles up” to its correct position.

```java
// Non-optimised bubble sort
public static int bubbleSortNonOptimised(ArrayList<Wine> list) {  4 usages
    int n = list.size();
    int swaps = 0;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1; j++) {
            if (list.get(j).alcohol() > list.get(j + 1).alcohol()) {
                Wine temp = list.get(j);
                list.set(j, list.get(j + 1));
                list.set(j + 1, temp);
                swaps++;
            }
        }
    }
    return swaps;
}
```

Code: src/main/java/Problem1_BubbleSort.java

The optimized variant introduces two optimizations:

- We know that after each pass, the largest unsorted element is placed at its correct position at the end, so the unsorted portion of the list decreases by 1. By introducing: `for (int j = 0; j < n - i - 1; j++)` we account for this and reduce the upper bound after each inner pass.

- Abort the outer loop with the swapped boolean. If no swaps have occurred, completing the remaining outer loop is pointless.

```java
// Optimised: exits early if no swaps happen in the outer pass, and reduces range for each inner pass
public static int bubbleSortOptimised(ArrayList<Wine> list) {  4 usages
    int n = list.size();
    int swaps = 0;

    for (int i = 0; i < n - 1; i++) {
        boolean swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (list.get(j).alcohol() > list.get(j + 1).alcohol()) {
                Wine temp = list.get(j);
                list.set(j, list.get(j + 1));
                list.set(j + 1, temp);
                swaps++;
                swapped = true;
            }
        }
        if (!swapped) break;
    }
    return swaps;
}
```

Code:src/main/java/Problem1_BubbleSort.java

3.2 Time Complexity

The bubblesort algorithm has a worst-case time complexity of $O(n^2)$ due to its nested for-loops. For each pass in the outer loop, the inner loop compares adjacent pairs of elements, and either swaps them or skips them before moving to the next pair. It will keep doing this until it has completed all passes through the list, even if no elements were swapped. The optimised bubble sort implementation checks each pass if any swaps were made, and exits early with the swapped bool. This can lead to a best-case time complexity of **Ω(n)** on sorted lists.

Optimized bubble sort time complexities:

- Best case: $\Omega(n)$
- Average case: $\Theta(n^2)$
- Worst case: $O(n^2)$

Unoptimized bubble sort time complexities:

- Best case: $\Omega(n^2)$
- Average case: $\Theta(n^2)$
- Worst case: $O(n^2)$

The output of our program demonstrates the significant difference in time efficiency between the optimized bubble sort on a sorted list (best-case) compared to the other scenarios.

The best-case only took 0.53 microseconds on average. This is because the expressed time complexity changes from linear $\Omega(n)$ to quadratic $\Theta(n^2)$ when comparing the last line of the results with the lines above.

However the time complexity does not change when comparing any bubble sort on the raw- versus the shuffled dataset. This is because the data is randomly ordered from the source, meaning a shuffle-operation only affects the time and number of swap operations, not the expressed time-complexity. They both present as average cases, where some elements are unsorted giving **$\Theta(n^2)$**.

```
Algorithm                    Input      Avg Time        Avg Operations
-------------------------------------------------------------------------------------
| Bubble Sort Non-optimized  raw        122 µs          2607 swaps                    |
| Bubble Sort Non-optimized  shuffled   38 µs           2677 swaps                    |
| Bubble Sort Non-optimized  sorted     27 µs           0 swaps                       |

| Bubble Sort Optimized      raw        111 µs          2607 swaps                    |
| Bubble Sort Optimized      shuffled   84 µs           2677 swaps                    |
| Bubble Sort Optimized      sorted     0,53 µs         0 swaps                       |
```

Image: reference_benchmark_results.txt

3.2.1 Calculating worst-case time complexity

The algorithm begins by storing the list size in a variable, which is a constant $O(1)$ operation. The outer for-loop iterates over every element in the list, giving it a linear $O(n)$ time complexity. The inner for-loop is what makes the bubble sort algorithm inefficient. Being nested inside the outer loop, it must traverse the list for each element visited by the outer loop. Since both loops have $O(n)$ complexity, their combination yields a total time complexity of $O(n) * O(n) =$ **$O(n^2)$.**

```
public static void bubbleSortNonOptimised(ArrayList<Wine> list) {
    int n = list.size(); +O(1)
    for (int i = 0; i < n - 1; i++) {  +n
        for (int j = 0; j < n - 1; j++) {  *n
            if (list.get(j).alcohol() > list.get(j + 1).alcohol()) {  + O(1)
                Wine temp = list.get(j);      +O(1)
                list.set(j, list.get(j + 1)); +O(1)
                list.set(j + 1, temp);        +O(1)
            }
        }
    }
}
```

O(1) + O(n) × (O(n) × (O(1) + O(1) + O(1) + O(1))) = O(5n²) → O(n²)

Image: images/Unoptimized_BubbleSort_Calculation.jpg

```
public static void bubbleSortOptimised(ArrayList<Wine> list) {
    int n = list.size();  +O(1)
    for (int i = 0; i < n - 1; i++) { +n
        boolean swapped = false;      +O(1)
        for (int j = 0; j < n - i - 1; j++) { *n
            if (list.get(j).alcohol() > list.get(j + 1).alcohol()) { +O(1)
                Wine temp = list.get(j);      +O(1)
                list.set(j, list.get(j + 1)); +O(1)
                list.set(j + 1, temp);        +O(1)
                swapped = true;               +O(1)
            }
        }
        if (!swapped) break;  +O(1)
    }
}
```

O(1) + O(n) × (O(1) + O(n) × (O(1) + O(1) + O(1) + O(1) + O(1)) + O(1))
= O(5n² + 2n + 1) → O(n²)

Image: images/Optimized_BubbleSort_Calculation.jpg

Note: Calculations were done before adding the swap-counter variable seen in the final implementation. As the counter only adds a constant variable, it does not affect the final calculation results.

4. Task 2 – Insertion Sort

4.1 Implementation

Insertion sort works by traversing the data structure comparing each element to its left. All elements to the left of the key (current element) are regarded as sorted, and the elements to the right unsorted. We assume the first element in the array is sorted, so we start with the element at index 1 as our first key. From here on we compare i[1] with i[0]. If our key is smaller than the value compared to on the left side, we shift all elements one position to the right, to open up the correct insertion slot in the ordered side for our current element. We continue traversing through the array, comparing our key against every element on the sorted side until we reach the end.

```java
public static int insertionSort(ArrayList<Wine> list) {  5 usages
    int comparisons = 0;
    // Start on the second index as the first is assumed to be sorted
    for (int i = 1; i < list.size(); i++) {
        comparisons++; // count the comparison in the for-loop
        Wine current_wine = list.get(i);
        double keyAlcohol = current_wine.alcohol();
        // Compare the current wine with each element to its left.
        // Shift elements with higher alcohol content one position to the right
        // until the correct insertion slot is found.
        int j = i - 1;
        while (j >= 0 && list.get(j).alcohol() > keyAlcohol) {
            list.set(j + 1, list.get(j));
            j--;
            comparisons++;
        }
        // set the current wine-element in the correct place in the sorted list
        list.set(j + 1, current_wine);
    }
    return comparisons;
}
```

Code: src/main/java/Problem2_InsertionSort.java

4.2 Time Complexity

Insertion Sort has the same time complexities as optimized bubblesort (**$O(n^2)$**, **$\Theta(n^2)$** and **$\Omega(n)$**) because of their shared characteristic: the nested loop. In this instance a while-loop. On a perfectly sorted list, for each element (*i*) the while loop checks the element to its left only once and immediately stops, because the condition `list.get(j).alcohol() > keyAlcohol` is false. Described as n × 1 = n total operations, that gives us a best case time complexity of **$\Omega(n)$**.

4.2.1 Calculating worst-case time complexity

```
public static int insertionSort(ArrayList<Wine> list) {  5 usages   mariusaellingsen *
    int comparisons = 0;

    for (int i = 1; i < list.size(); i++) {                          *n

        comparisons++;                                               +O(1)
        Wine current_wine = list.get(i);                             +O(1)
        double keyAlcohol = current_wine.alcohol();                  +O(1)
        int j = i - 1;                                               +O(1)

        while (j >= 0 && list.get(j).alcohol() > keyAlcohol) {       *n
            list.set(j + 1, list.get(j));                            +O(1)
            j--;                                                     +O(1)
            comparisons++;                                           +O(1)
        }
        list.set(j + 1, current_wine);                               +O(1)
    }
    return comparisons;
}
```

$O(n) \times (O(1) + O(1) + O(1) + O(1) + O(n) \times (O(1) + O(1) + O(1)) + O(1)) = O(2n^2 + 4n) \rightarrow O(n^2)$

Image: images/InsertionSort_Calculation.jpg

Similarly to bubble sort the time complexity remains $O(n^2)$ when using the algorithm on the dataset before and after shuffling it, since it's randomly ordered from the source. Comparing a shuffled list with a sorted list is how we caused a shift in expressed time complexity in the benchmarking: (**$\Theta(n^2)$** for raw and shuffled, and **$\Omega(n)$** for sorted)

```
| Insertion Sort              raw         59 µs          2717 comparisons
| Insertion Sort              shuffled    57 µs          2787 comparisons
| Insertion Sort              sorted      3 µs           110 comparisons
```

Image: reference_benchmark_results.txt

5. Task 3 – Merge Sort

5.1 Implementation

Merge sort is a divide and conquer algorithm. It splits the list(s) in half until each sublist only contains a single element. Once the lists are separated, they are ready for merging. The algorithm then takes each sublist and merges them together in sorted order. This method of sorting is efficient on larger data sets and for sorting linked lists. It's widely used due to its reliability as it guarantees a time complexity of **O(*n* log *n*)** across all cases.

```java
// Wrapper method so our benchmark tool can call the recursive method
public static int mergeSort(ArrayList<Wine> list) { return mergeSort(list, left: 0, right: list.size() - 1); }

// Method only containing splitting logic
private static int mergeSort(ArrayList<Wine> list, int left, int right) {  3 usages  mariusaellingsen +1
    if (left >= right) return 0;                    // If there is only 1 element, already sorted. Return.
    int mid = left + (right - left) / 2;            // Find the middle
    int mergeCounts = mergeSort(list, left, mid)    // Sort everything left of mid until it hits single elements
            + mergeSort(list, left: mid + 1, right);  // Sort everything right of mid the same way
    mergeCounts += merge(list, left, mid, right);   // Merge sorted halves and count the operation
    return mergeCounts;
}
```

Code: src/main/java/Problem3_MergeSort.java

Our implementation uses index pointers (left, mid, right) rather than creating sub-lists. Instead of copying data into arrays, the algorithm works directly on the original list by calculating a midpoint index and recursively sorting each half in place. During the merge step, a temporary list is used to hold the merged result, which is then written back into the original list at the correct position. This avoids the memory overhead of allocating new sub-arrays at every recursive call.

```java
// Merges two sorted halves into a single sorted list
private static int merge(ArrayList<Wine> list, int left, int mid, int right) {  1 usage  mariusaellingsen
    List<Wine> temp = new ArrayList<>();            // Temporary list for the sorted result before writing back
    int i = left, j = mid + 1;
    while (i <= mid && j <= right) {                // Compares current element of each half, adds the smaller one to temp
        if (list.get(i).alcohol() <= list.get(j).alcohol()) {
            temp.add(list.get(i++));                // Take the smallest value from left half
        } else {
            temp.add(list.get(j++));                // Take the smallest value from the right half
        }
    }
    while (i <= mid) temp.add(list.get(i++));       // Checks for leftover elements in left half, append to temp
    while (j <= right) temp.add(list.get(j++));     // Checks for leftover elements in right half, append to temp
    for (int k = 0; k < temp.size(); k++) {         // Write the sorted temp list back into the original list
        list.set(left + k, temp.get(k));
    }
    return 1;                                       // Count this merge operation
}
```

Code: src/main/java/Problem3_MergeSort.java

5.2 Number of merges on shuffled- and raw dataset

The amount of merges performed by merge sort is not affected by input order, only input size, as reflected in the benchmark difference between “all” and the other scenarios.

```
| Merge Sort                  raw         22 µs          110 merge operations
| Merge Sort                  shuffled    21 µs          110 merge operations
| Merge Sort                  sorted      19 µs          110 merge operations
| Merge Sort                  all         2,12 ms        6496 merge operations
```

Image: reference_benchmark_results.txt

This is because merge sort always divides the list into halves and merges them back together regardless of whether the data is already sorted, randomly ordered, or completely reversed. It has no early exit mechanism like the swapped flag in bubble sort or the while-loop condition in insertion sort.

6. Task 4 – Quick Sort

6.1 Implementation

Quick sort is also a divide and conquer algorithm. It works by selecting a pivot element and partitioning the list into two sides: elements smaller than the pivot to the left, and elements larger to the right. The pivot is then in its final sorted position, and the algorithm recurses on each side until the entire list is sorted.
Our implementation separates the logic into two functions: quickSort handles the recursive divide step, and partition handles the pivot selection and rearrangement.
The base case *if (low >= high) return* handles single-element sublists, which are already sorted by definition, mirroring the same principle as in merge sort.

```
private static int quickSort(ArrayList<Wine> list, int low, int high, String pivotType) {  6 usages
    if (low >= high) return 0; // Base case: only 1 element, already sorted
    int[] result = partition(list, low, high, pivotType); // [pivotIndex, count]
    int pivotIndex = result[0];
    int count = result[1];
    count += quickSort(list, low,  high: pivotIndex - 1, pivotType);  // Sort left of pivot
    count += quickSort(list,  low: pivotIndex + 1, high, pivotType); // Sort right of pivot
    return count;
}
```

Code: src/main/java/Problem4_QuickSort.java

A key design choice in quick sort is pivot selection, as it determines how evenly the list is split each recursion. Our implementation supports four strategies, all of which move the chosen pivot to the end of the sublist before partitioning, so the partitioning logic remains identical regardless of strategy:

```java
// Returns int[] where [0] = pivot's final index, [1] = comparison count
private static int[] partition(ArrayList<Wine> list, int low, int high, String pivotType) {  1 usage
    // Move the chosen pivot to the end before partitioning
    switch (pivotType) {
        case "first" ->
                Collections.swap(list, low, high);
        case "random" -> {
            int randomIndex = low + new Random().nextInt( bound: high - low + 1);
            Collections.swap(list, randomIndex, high);
        }
        case "median" -> {
            int mid = low + (high - low) / 2;
            double first = list.get(low).alcohol();
            double middle = list.get(mid).alcohol();
            double last = list.get(high).alcohol();
            // Find which of the three values is the median and swap it to the end
            if ((first <= middle && middle <= last) || (last <= middle && middle <= first)) {
                Collections.swap(list, mid, high);
            } else if ((middle <= first && first <= last) || (last <= first && first <= middle)) {
                Collections.swap(list, low, high);
            }
        }
        case "last" -> {
            // Do nothing - last element is already in position
        }
    }

    double pivot = list.get(high).alcohol();
    int i = low - 1, comparisons = 0;

    // Walk through the section, moving elements smaller than pivot to the left
    for (int j = low; j < high; j++) {
        comparisons++; // Count every comparison
        if (list.get(j).alcohol() < pivot) {
            i++;
            Collections.swap(list, i, j);
        }
    }

    // Place pivot in its final sorted position
    Collections.swap(list,  i: i + 1, high);
    return new int[]{i + 1, comparisons}; // Return pivot index and comparison count
}
```

Code: src/main/java/Problem4_QuickSort.java

- **First** - always selects the first element. Simple, but performs poorly on already sorted data as it consistently produces the worst-case split.
- **Last** - selects the last element. The default; no swap needed.
- **Random** - selects a random element. The best general-purpose choice as it makes worst-case behaviour unlikely.
- **Median-of-three** - selects the median of the first, middle, and last elements. A good balance between predictability and performance, avoiding the pitfalls of first/last on sorted data.

Once the pivot is moved to the end, the partition function walks through the sublist with a pointer j. Any element smaller than the pivot is swapped to the left side by incrementing i and swapping. After the full pass, the pivot is placed at its final sorted position i + 1 and its index is returned so quickSort can recurse on both sides.

The pivot strategies are implemented using a switch expression instead of if-else, as it more clearly maps each strategy to its own case, making the code easier to read and expand without compromising the functionality of the algorithm

6.2 Comparisons and pivot strategies

As demonstrated in the benchmarks, the number of comparisons vary depending on which pivot strategy we use.

```
| Quick Sort - First pivot     raw        3 µs           740 comparisons                 |
| Quick Sort - First pivot     shuffled   3 µs           735 comparisons                 |
| Quick Sort - First pivot     sorted     13 µs          6105 comparisons                |
| Quick Sort - First pivot     all        1,28 ms        607317 comparisons              |

| Quick Sort - Last pivot      raw        3 µs           685 comparisons                 |
| Quick Sort - Last pivot      shuffled   3 µs           714 comparisons                 |
| Quick Sort - Last pivot      sorted     22 µs          6105 comparisons                |
| Quick Sort - Last pivot      all        1,28 ms        609957 comparisons              |

| Quick Sort - Random pivot    raw        8 µs           744 comparisons                 |
| Quick Sort - Random pivot    shuffled   8 µs           736 comparisons                 |
| Quick Sort - Random pivot    sorted     6 µs           742 comparisons                 |
| Quick Sort - Random pivot    all        1,56 ms        606812 comparisons              |

| Quick Sort - Median pivot    raw        3 µs           665 comparisons                 |
| Quick Sort - Median pivot    shuffled   3 µs           656 comparisons                 |
| Quick Sort - Median pivot    sorted     2 µs           546 comparisons                 |
| Quick Sort - Median pivot    all        1,27 ms        607954 comparisons              |
================================================================================================
```

Image: reference_benchmark_results.txt

The median pivot strategy results in fewest comparisons with our dataset. This is a result of the nature of our data and the random-pivot not hitting a statistically unlikely optimal route. However when accounting for our benchmark running each algorithm 100 times, we can appreciate the unlikelihood of this occurring.

The median pivot can perform poorly in datasets which contain a lot of duplicates, which makes it suited for our dataset given our preprocessing. We attempted to test this by also including the benchmarks for ALL. In this scenario it's still the fastest algorithm in time measurements, but it now has more comparisons than random- and first pivot. This suggests that the duplicates in the ALL dataset reduces its efficiency.

For the purpose of our dataset it is clear that the median pivot is the best choice of the quick-sort variants.

7. Conclusion

Bubble sort unsurprisingly has the worst outcome at 122 µs and 111 µs on raw data, consistent with its reputation as a learning tool rather than a practical algorithm. The optimised variant achieves 0.53 µs on sorted data, but this relies entirely on already-sorted input and offers no advantage otherwise.

```
================================================================================================
|                          Sorting algorithm performance comparison                            |
================================================================================================
Algorithm                      Input      Avg Time        Avg Operations
------------------------------------------------------------------------------------------------
| Bubble Sort Non-optimized    raw        122 µs          2607 swaps                          |
| Bubble Sort Non-optimized    shuffled   38 µs           2677 swaps                          |
| Bubble Sort Non-optimized    sorted     27 µs           0 swaps                             |

| Bubble Sort Optimized        raw        111 µs          2607 swaps                          |
| Bubble Sort Optimized        shuffled   84 µs           2677 swaps                          |
| Bubble Sort Optimized        sorted     0,53 µs         0 swaps                             |

| Insertion Sort               raw        59 µs           2717 comparisons                    |
| Insertion Sort               shuffled   57 µs           2787 comparisons                    |
| Insertion Sort               sorted     3 µs            110 comparisons                     |
| Insertion Sort               all        19,85 ms        9242963 comparisons                 |

| Merge Sort                   raw        22 µs           110 merge operations                |
| Merge Sort                   shuffled   21 µs           110 merge operations                |
| Merge Sort                   sorted     19 µs           110 merge operations                |
| Merge Sort                   all        2,12 ms         6496 merge operations               |

| Quick Sort - First pivot     raw        3 µs            740 comparisons                     |
| Quick Sort - First pivot     shuffled   3 µs            735 comparisons                     |
| Quick Sort - First pivot     sorted     13 µs           6105 comparisons                    |
| Quick Sort - First pivot     all        1,28 ms         607317 comparisons                  |

| Quick Sort - Last pivot      raw        3 µs            685 comparisons                     |
| Quick Sort - Last pivot      shuffled   3 µs            714 comparisons                     |
| Quick Sort - Last pivot      sorted     22 µs           6105 comparisons                    |
| Quick Sort - Last pivot      all        1,28 ms         609957 comparisons                  |

| Quick Sort - Random pivot    raw        8 µs            744 comparisons                     |
| Quick Sort - Random pivot    shuffled   8 µs            736 comparisons                     |
| Quick Sort - Random pivot    sorted     6 µs            742 comparisons                     |
| Quick Sort - Random pivot    all        1,56 ms         606812 comparisons                  |

| Quick Sort - Median pivot    raw        3 µs            665 comparisons                     |
| Quick Sort - Median pivot    shuffled   3 µs            656 comparisons                     |
| Quick Sort - Median pivot    sorted     2 µs            546 comparisons                     |
| Quick Sort - Median pivot    all        1,27 ms         607954 comparisons                  |
================================================================================================
```

Image: reference_benchmark_results.txt

Insertion sort performs well on sorted data where it achieves 3 µs and 110 comparisons due to its Ω(n) best case. On raw and shuffled data the $O(n^2)$ average case becomes apparent, and the ALL benchmark reveals the true cost at scale: 19.85 ms and over 9 million comparisons.

Merge sort is consistent across all input types, always performing exactly 110 merge operations on the unique dataset and only 6496 on the full dataset regardless of input order. It carries more overhead than quick sort and insertion sort at n=111, but the ALL benchmark demonstrates where it belongs: 2.12 ms versus insertion sort's 19.85 ms confirms that O(n log n) scales far better than $O(n^2)$ on larger inputs.

Considering the three criteria laid out in the introduction: time measurements, comparison counts and theory, **quick sort with median pivot** is the best choice for our dataset. It has the fewest comparisons (among its variants): 665 raw. Time measurements are among the lowest: 3 µs. Theory supports this too: median pivot is expected to produce near-optimal splits on a datasets such as ours that consists of unique values. Quick sort with median pivot was therefore selected as the algorithm of choice when we create the sorted list for benchmarking in the beginning of the program.

8. Additional Notes

8.1 Task 3 – Extra Note on Merge Sort Time Complexity

Because merge sort always divides the list into halves and merges them back together, that also means that merge sort does not achieve Ω(n) even in the best-case scenario, unlike bubble- and insertion sort. Therefore it's always **Θ(n log n)** regardless of input order.

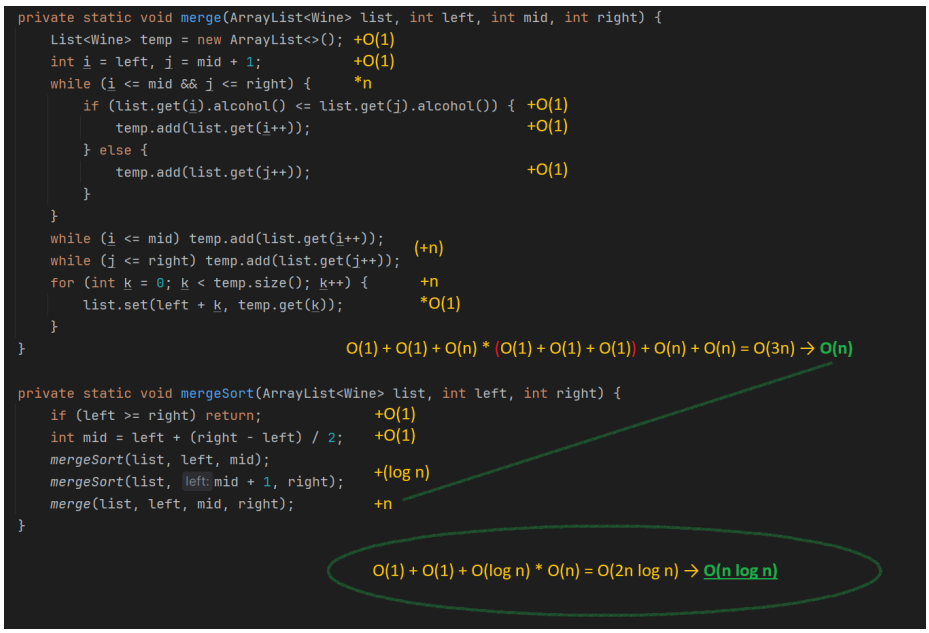

Image: images/Mergesort_Calculation.jpg - Calculating worst-case time complexity.

Note: Calculations were done before adding the counter variable seen in the final implementation. As the counter only adds a constant variable, it does not affect the final calculation results.

Merge sort's O(n log n) advantage over insertion sort's O(n^2) becomes significant at larger dataset sizes, making it the better general-purpose choice for larger inputs. This is especially demonstrated in the time difference of the two “ALL”-type measurements.

```
| Insertion Sort              raw         59 µs          2717 comparisons              |
| Insertion Sort              shuffled    57 µs          2787 comparisons              |
| Insertion Sort              sorted      3 µs           110 comparisons               |
| Insertion Sort              all         19,85 ms       9242963 comparisons           |

| Merge Sort                  raw         22 µs          110 merge operations          |
| Merge Sort                  shuffled    21 µs          110 merge operations          |
| Merge Sort                  sorted      19 µs          110 merge operations          |
| Merge Sort                  all         2,12 ms        6496 merge operations         |
```

Image: reference_benchmark_results.txt

Insertion sort outperforms merge sort on the sorted dataset, but otherwise is less efficient. This is due to merge sort's inability to exit early, while insertion sort does in the while-loop condition, achieving Ω(n) where merge sort always will be Ω(n log n).

8.2 Task 4 – Extra note on Quick Sort Time Complexity

The outer **O(n)** factor represents the recursion depth, which in the worst case reaches n because each partition produces one sublist of n-1 elements and one of 0. In the average case however, the pivot divides the list roughly in half each time, reducing the recursion depth from O(n) to O(log n), making the formula instead:

$$O(\log n) \times (O(1) + O(n) + O(n) + O(n)) = O(n \log n)$$

Calculating worst-case time complexity:

```
private static void quickSort(ArrayList<Wine> list, int low, int high, String pivotType) {  +O(n)
    if (low >= high) return;                                                                 +O(1)
    int pivotIndex = partition(list, low, high, pivotType);                                  +O(n)
    quickSort(list, low, high: pivotIndex - 1, pivotType);                                   +O(n)
    quickSort(list, low: pivotIndex + 1, high, pivotType);                                   +O(n)        O(n) × (O(1) + O(n) + O(n) + O(n) ) = O(n²)
}
```

Image: Quicksort_Calculation.jpg - The partition method only consists of constants, except for the for-loop resulting in O(n).

Note: Calculations were done before adding the counter variable seen in the final implementation. As the counter only adds a constant variable, it does not affect the final calculation results.

8.3 Space Complexities

Beyond time complexity and counting operations, it is worth considering how much additional memory each algorithm requires during execution. Unlike time complexity, space complexity measures the extra memory allocated on top of the input itself.

Bubble Sort has a space complexity of O(*1*). It sorts in place using only a fixed number of variables (n, swaps, swapped, temp), regardless of input size.

Insertion Sort also operates in place with O(*1*) space complexity. The only additional memory used is for a few local variables: the current element, its alcohol value, and the index pointer j.

Merge Sort has a space complexity of O(*n*). While it avoids allocating new sublists by using index pointers, the merge step still requires a temporary list to hold the merged result before writing it back. This temporary list grows proportionally with the input size.

Quick Sort has an average space complexity of O(*log n*) due to the recursive call stack. Each recursive call adds a stack frame, and with a well-chosen pivot producing balanced splits, the recursion depth stays at O(*log n*). In the worst case such as a sorted list with first or last pivot, this degrades to O(*n*).

8.4 Program architecture

The project is structured as a single module under src/main/java, with Main as the entry point. It handles configuration (warmup rounds, test rounds, export flag) and orchestrates the full benchmarking run. It prepares four dataset variants; RAW, SHUFFLED, SORTED, and ALL, using WineLoader, which parses the two CSV files and exposes both a duplicate-filtered (loadUniqueWines) and unfiltered (loadAllWines) version of the dataset.

Since the wine CSV files use semicolons rather than commas as delimiters, WineLoader uses BufferedReader with a manual split(";") instead of a standard CSV library, keeping the parsing lightweight and avoiding an unnecessary dependency.

```
private static ArrayList<Wine> loadWinesUnfiltered(String filePath, Wine.WineType type) {
    ArrayList<Wine> wines = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
        String line;
        boolean firstLine = true;
        while ((line = br.readLine()) != null) {
            if (firstLine) { firstLine = false; continue; }
            String[] parts = line.split( regex: ";");
            if (parts.length < 12) continue;
            wines.add(new Wine(
                    Double.parseDouble(parts[0]),
                    Double.parseDouble(parts[1]),
                    Double.parseDouble(parts[2]),
                    Double.parseDouble(parts[3]),
                    Double.parseDouble(parts[4]),
                    Double.parseDouble(parts[5]),
                    Double.parseDouble(parts[6]),
                    Double.parseDouble(parts[7]),
                    Double.parseDouble(parts[8]),
                    Double.parseDouble(parts[9]),
                    Double.parseDouble(parts[10]),
                    Integer.parseInt(parts[11].trim()),
                    type
            ));
```

The Wine record holds all 12 properties per entry plus a WineType enum distinguishing red from white. While we only utilize a single property *alcohol* in this program, we kept the wine-abstraction out of personal preference and as a theoretical scope expansion measure.

Code:src/main/java/WineLoader.java

Each sorting algorithm is isolated in its own class (Problem1_BubbleSort through Problem4_QuickSort), all sharing the same signature: they accept an ArrayList<Wine> and return an integer operation count. This uniform interface allows Main to pass algorithm references as Function<ArrayList<Wine>, Integer> lambdas directly into BenchmarkHandler.

```
public static void jvmWarmup(int warmupRounds, ArrayList<Wine> raw,
                             ArrayList<Wine> shuffled, ArrayList<Wine> sorted,
                             Function<ArrayList<Wine>, Integer> algorithm) {
    for (int i = 0; i < warmupRounds; i++) {
        algorithm.apply(new ArrayList<>(raw));
        algorithm.apply(new ArrayList<>(shuffled));
        algorithm.apply(new ArrayList<>(sorted));
    }
}
```

Code:src/main/java/BenchmarkHandler.java

BenchmarkHandler handles both JVM warm-up and the actual benchmarking. Each benchmark run creates a fresh copy of the list to avoid measuring an already-sorted input:

```
public static BenchmarkResult benchmark(String name, int testRounds, InputType inputType,
                                        ArrayList<Wine> wines, Function<ArrayList<Wine>, Integer> algorithm,
                                        OperationLabel operationLabel) {
    Timer timer = new Timer();
    int totalCount = 0;
    for (int i = 0; i < testRounds; i++) {
        ArrayList<Wine> list = new ArrayList<>(wines);
        timer.start();
        totalCount += algorithm.apply(list);
        timer.stop();
    }
    return new BenchmarkResult(name, inputType, timer.averageMicros(), avgOperations: totalCount / testRounds, operationLabel);
}
```

Code:src/main/java/BenchmarkHandler.java

and uses Timer to accumulate nanosecond timings across all test rounds:

```java
public class Timer {
    private long endTime;
    private long totalTime;
    private int laps;

    public void start() { startTime = System.nanoTime(); }

    public void stop() {  1 usage
        endTime = System.nanoTime();
        totalTime += (endTime - startTime); // store raw nanoseconds
        laps++;
    }

    public double totalMillis() { return totalTime / 1_000_000.0; // convert only for display }

    public double averageMillis() {
        return laps == 0 ? 0 : ((double) totalTime / laps) / 1_000_000.0;
    }

    public double totalMicros() {
        return totalTime / 1_000.0;
    }

    public double averageMicros() {
        return laps == 0 ? 0 : ((double) totalTime / laps) / 1_000.0;
    }

    public void reset() {
        startTime = 0;
        endTime = 0;
        totalTime = 0;
        laps = 0;
    }
}
```

Code:src/main/java/Timer.java

Results are collected as BenchmarkResult records, which bundle the algorithm name, InputType, average time in microseconds, average operation count, and an OperationLabel enum indicating what was counted (swaps, comparisons, or merge operations).

```java
public record BenchmarkResult(  9 usages
        String name,  3 usages
        InputType inputType,  1 usage
        double avgMicros,  1 usage
        long avgOperations,  1 usage
        OperationLabel operationLabel  1 usage
) {}
```

```java
public enum OperationLabel {  35 usages
    COMPARISONS("comparisons"),  20 usages
    SWAPS("swaps"),  6 usages
    MERGE_OPERATIONS("merge operations");  4 usages

    private final String label;  2 usages

    OperationLabel(String label) { this.label = label; }

    public String getLabel() { return label; }
}
```

Code:src/main/java/records/Benchmarkresult.java *Code:src/main/java/enums/OperationLabel.java*

Finally, PrintHandler formats and prints the results as a table, and can optionally export them to benchmark_results.txt in the project root. This feature provides the examiner with direct access to the benchmark report analyzed in this paper. (reference_benchmark_results.txt)

```
private static String formatResults(ArrayList<BenchmarkResult> results) {  1 usage
    int width = 92;
    String border = "=".repeat(width);
    String divider = "-".repeat(width);
    String title = "Sorting algorithm performance comparison";
    int padding = (width - 2 - title.length()) / 2;
    String centeredTitle = "|" + " ".repeat(padding) + title + " ".repeat( count: width - 2 - padding - title.length()) + "|";

    StringBuilder sb = new StringBuilder();
    sb.append("\n").append(border).append("\n");
    sb.append(centeredTitle).append("\n");
    sb.append(border).append("\n");
    sb.append(String.format("%-30s %-10s %-15s %-25s%n", "Algorithm", "Input", "Avg Time", "Avg Operations"));
    sb.append(divider).append("\n");

    String lastName = "";
    for (BenchmarkResult r : results) {
        if (!r.name().equals(lastName)) {
            if (!lastName.isEmpty()) sb.append("\n");
            lastName = r.name();
        }
        sb.append(String.format("| %-28s %-10s %-15s %-33s|%n",
                r.name(),
                r.inputType().getLabel(),
                formatTime(r.avgMicros()),
                r.avgOperations() + " " + r.operationLabel().getLabel()));
    }
    sb.append(border).append("\n");
    return sb.toString();
}
```

Code:src/main/java/records/PrintHandler.java

```
public static void printResults(ArrayList<BenchmarkResult> results, boolean exportToFile) {  1 usage   kremfote
    String output = formatResults(results);
    System.out.println(output);
    if (exportToFile) {
        try (PrintWriter writer = new PrintWriter(new FileWriter( fileName: "benchmark_results.txt"))) {
            writer.println(output);
            System.out.println("exportToFile-flag set to TRUE.");
            System.out.println("Results saved in project root. ");
        } catch (IOException e) {
            System.err.println("Error writing results to file.");
        }
    }
    else {System.out.println("exportToFile-flag set to FALSE. Results are not saved. Set to TRUE in main if need to save results locally.");}
}
```

Code:src/main/java/records/PrintHandler.java

The class also applies formatting rules, such as adaptive time unit selection (microseconds vs. milliseconds) based on magnitude, and conditional decimal precision for improved readability of small values.

```
private static String formatTime(double micros) {
    if (micros >= 1000) return String.format("%.2f ms", micros / 1000);
    if (micros >= 1)    return String.format("%.0f µs", micros);
    return String.format("%.2f µs", micros);
}
```

Code:src/main/java/records/PrintHandler.java

8.5 Reflections

Given the format of the exam we defer this section to the video attachment: video_demonstration

9. Bibliography

IBM. (2024, February 12). The JIT compiler. IBM SDK, Java Technology Edition 8.

https://www.ibm.com/docs/en/sdk-java-technology/8?topic=reference-jit-compiler

Paulo Cortez, A. C. (2009). *Wine Quality* [Data set]. UCI Machine Learning Repository.

https://archive.ics.uci.edu/dataset/186/wine+quality